

Assistant Conversationnel pour l'Analyse de Repository Logiciel

Prototype développé sur le projet Rafiki

Architecture Hybride

Chatbot Conversationnel + Génération de Rapport

LangChain | ChromaDB | Sentence-Transformers

NVIDIA Nemotron | Groq | DeepSeek

Cross-Encoder | Anti-Hallucination Multi-Niveaux

Exercice de Sélection

Assistant Conversationnel du Repository

ELOUAKATE Saad

saadelouakate7@gmail.com

GitHub : SaadEl-bit/Repository-Assistant

Table des matières

1	Introduction et Contexte	3
1.1	Présentation du Projet Rafiki	3
1.2	Objectif du Prototype	3
1.3	Schéma Architectural Global	3
2	Analyse du Repository	3
2.1	Parcours Récursif des Fichiers	3
2.2	Filtrage et Classification	3
2.2.1	Fichiers Inclus	3
2.2.2	Fichiers Exclus	4
2.3	Parsing Sémantique par Type de Fichier	4
2.4	Extraction de l'Arborescence	4
3	Indexation des Fichiers, du Code et de la Documentation	4
3.1	Génération des Chunks	5
3.2	Choix du Modèle d'Embedding	5
3.3	Stockage dans ChromaDB	5
3.3.1	Paramètres de Stockage	5
3.3.2	Structure de la Collection	5
4	Recherche des Informations Pertinentes	6
4.1	Premier Niveau : Filtrage Cosine	6
4.2	Deuxième Niveau : Cross-Encoder (Re-Ranking)	6
4.2.1	Pourquoi 50 candidats au lieu de 3 ?	6
4.2.2	Évolution du seuil	7
5	Génération d'une Réponse Fiable et Sourced	7
5.1	Construction du Prompt RAG	7
5.2	Cascade de Fournisseurs LLM	7
5.3	Mémoire Conversationnelle	8
5.4	Citation des Sources	8
6	Limitation des Hallucinations	8
6.1	Filtrage Cosine Large (Premier Passage)	8
6.2	Cross-Encoder (Deuxième Passage, le Plus Précis)	8
6.3	Contrainte Comportementale par Prompt	9
6.4	Message de Fallback	9
6.5	Limites Assumées	9
7	Identification des Zones Critiques du Projet	9
7.1	Fichiers Sensibles	9
7.2	Complexité Cyclomatique	9
7.3	Modules Non Testés	10
7.4	Accès Utilisateur	10
8	Évaluation de la Qualité de l'Assistant	10

8.1	Tests de Validation par Phase	10
8.2	Évaluation Structurée	10
8.3	Résultats des Tests	11
8.4	Axes d'Amélioration	11
9	Usage de l'IA	11
9.1	Contributions de l'IA	11
9.1.1	Brainstorming et Conception	11
9.1.2	Usage du LLM dans le Prototype	12
9.2	Contributions Personnelles	12
9.2.1	Décisions Techniques	12
9.2.2	Implémentation	12
9.2.3	Analyse Critique	12
9.3	Limites de l'Aide IA	12
10	Conclusion	13
10.1	Forces du Prototype	13
10.2	Axes d'Amélioration	13

1 Introduction et Contexte

1.1 Présentation du Projet Rafiki

Rafiki est un projet personnel d'accompagnement des étudiants de deuxième année du baccalauréat. Le repository contient l'ensemble du code source, de la documentation et des configurations nécessaires au fonctionnement de la plateforme.

1.2 Objectif du Prototype

Le prototype développé vise à construire un **assistant conversationnel hybride** capable d'analyser le repository Rafiki et de répondre en langage naturel aux questions des utilisateurs. Le scope de la phase 1 se concentre sur :

- Un **chatbot conversationnel** avec mémoire et citations de sources ;
- Une **architecture évolutive** prévue pour intégrer ultérieurement la génération de rapports structurés (.md).

1.3 Schéma Architectural Global

Le pipeline de l'assistant suit un flux en trois phases :

1. **Ingestion** : parcours, parsing et chunking sémantique du repository ;
2. **Indexation** : vectorisation et stockage dans ChromaDB ;
3. **Interaction** : recherche RAG + génération de réponse via LLM.

2 Analyse du Repository

2.1 Parcours Récursif des Fichiers

Le système parcourt l'arborescence du repository à l'aide de la fonction `discover_files()` implémentée dans le module `config.py`. Ce parcours est récursif et identifie l'ensemble des fichiers présents.

2.2 Filtrage et Classification

Les fichiers sont différenciés en deux catégories.

2.2.1 Fichiers Inclus

Les extensions suivantes sont prises en charge, conformément à la stack technique du projet Rafiki :

- .py — fichiers Python (code source) ;
- .md — fichiers Markdown (documentation) ;

- `.json` — fichiers de configuration ;
- `.js`, `.css` — fichiers frontend (le cas échéant).

2.2.2 Fichiers Exclus

Par mesure de sécurité et d’optimisation, les éléments suivants sont exclus dès la phase de découverte :

- Fichiers binaires et compilés ;
- Fichiers `.env` (variables d’environnement sensibles) ;
- Répertoire `chroma_db/` (base vectorielle locale) ;
- Clés API, tokens et informations d’identification.

Note. — Les fichiers sensibles sont traités séparément par le module `scan_sensitive()` afin d’éviter toute indexation accidentelle dans la base vectorielle.

2.3 Parsing Sémantique par Type de Fichier

Le système adapte sa stratégie de parsing en fonction du type de fichier :

Type de fichier	Méthode	Unité de chunking
Code source (<code>.py</code>)	AST Python (<code>ast</code>)	Fonction, classe ou bloc logique
Documentation (<code>.md</code>)	Parsing par sections	Section délimitée par <code>##</code>
Configuration (<code>.json</code>)	Lecture intégrale	Fichier complet (configurations généralement courtes)

TABLE 1 – Stratégies de parsing et de chunking par type de fichier.

Principe fondamental. — Le chunking sémantique préserve l’intégrité des unités logiques (fonction entière, classe complète) afin d’éviter une fragmentation nuisible à la qualité de la recherche vectorielle.

2.4 Extraction de l’Arborescence

La fonction `extract_repo_tree()` génère une représentation hiérarchique du repository, utile pour :

- Comprendre la structure générale du projet ;
- Identifier les modules principaux ;
- Proposer un parcours de lecture aux nouveaux développeurs.

3 Indexation des Fichiers, du Code et de la Documentation

3.1 Génération des Chunks

L'ensemble des chunks extraits par le parsing AST s'élève à **812 unités** pour le repository Rafiki (dont 1 arbre d'architecture). Chaque chunk est enrichi de métadonnées structurées :

Listing 1 – Structure des métadonnées par chunk.

```

1 {
2   "source": "src/auth.py",
3   "type": "function",
4   "name": "authenticate_user",
5   "lines_start": 45,
6   "lines_end": 62,
7   "language": "python",
8   "docstring": "Verifie les credentials et retourne un token JWT"
9 }
```

3.2 Choix du Modèle d'Embedding

Le système utilise le modèle **paraphrase-multilingual-MiniLM-L12-v2** pour la vectorisation. Ce choix résulte d'une itération technique :

Modèle testé	Résultat	Motif du rejet
all-MiniLM-L6-v2	Échec	Monolingue anglais; distances de retrieval > 0,5
paraphrase-multilingual-MiniLM-L12-v2	Succès	Support multilingue; distances 0,34–0,59

TABLE 2 – Itération sur le choix du modèle d'embedding.

Avantage clé. — L'utilisation d'un modèle local élimine la dépendance à une API externe pour la phase d'embedding.

3.3 Stockage dans ChromaDB

3.3.1 Paramètres de Stockage

- **Taille de batch** : 100 chunks par batch, afin d'éviter la saturation mémoire;
- **Persistance** : stockage local dans le répertoire `chroma_db/`;
- **Reconstruction** : paramètre `force_rebuild=True` pour une reconstruction complète; sinon, récupération des collections existantes.

3.3.2 Structure de la Collection

Chaque document stocké dans ChromaDB contient :

- **documents** : le contenu textuel du chunk ;
- **embeddings** : le vecteur numérique (384 dimensions) ;
- **metadatas** : les métadonnées structurées (source, type, lignes, etc.) ;
- **ids** : identifiant unique généré automatiquement.

4 Recherche des Informations Pertinentes

La recherche s'appuie sur un pipeline à **deux niveaux** : un filtrage cosine large, puis un ré-ordonnancement fin par cross-encoder.

4.1 Premier Niveau : Filtrage Cosine

1. **Vectorisation de la question** : le modèle d'embedding transforme la question en vecteur numérique ;
2. **Requête ChromaDB** : la méthode `similarity_search_with_score()` calcule les distances cosinus ;
3. **Filtrage large** : on conserve les chunks avec une distance $< 0,80$. Ce seuil volontairement permissif (au lieu de 0,65) permet de remonter 50 candidats sans jeter de bons résultats trop tôt.

4.2 Deuxième Niveau : Cross-Encoder (Re-Ranking)

4. **Un cross-encoder** (`cross-encoder/ms-marco-MiniLM-L-6-v2`) compare chaque paire (question, chunk) et attribue un score logit. Contrairement au calcul cosine qui compare des vecteurs, le cross-encoder lit réellement le texte et évalue si le chunk répond à la question ;
5. **Filtrage fin** : les chunks avec un score négatif sont rejetés ;
6. **Envoi au LLM** : les chunks retenus sont passés comme contexte, avec 2000 caractères maximum par chunk.

4.2.1 Pourquoi 50 candidats au lieu de 3 ?

Au départ, on envoyait 5 chunks directement au LLM, mais on a eu une erreur 413 (*Request too large*) avec Groq à cause de l'historique qui s'ajoutait. Plus tard, en ajoutant le cross-encoder, on s'est rendu compte qu'avec seulement 3 candidats, il rejetait tout — pas assez de matière pour faire un tri. Solution : remonter largement (50 chunks, seuil 0,80), et laisser le cross-encoder décider.

4.2.2 Évolution du seuil

Version	Seuil	Justification
Initiale (modèle anglais)	0,50	Trop restrictif, rien ne passait
Modèle multilingue seul	0,65	Équilibre correct
Actuelle (avec cross-encoder)	0,80	Le cross-encoder fait le filtrage fin ; le seuil cosine sert juste à éviter 500 candidats inutiles

TABLE 3 – Évolution du seuil de pertinence au fil des itérations.

5 Génération d'une Réponse Fiable et Sourcede

5.1 Construction du Prompt RAG

Le prompt système, défini dans le module `rag_chain.py`, encadre strictement le comportement du LLM :

Listing 2 – Template du prompt système.

```

1 Tu es un assistant expert en analyse de repository logiciel.
2 Reponds UNIQUEMENT sur la base du CONTEXTE fourni.
3 Si le contexte ne permet pas de répondre, dis explicitement :
4 "Je ne dispose pas d'informations suffisantes dans le
5 repository pour répondre."
6 Cite TOUJOURS tes sources : nom du fichier et lignes
7 concernées.
8 Ne fais aucune supposition hors du contexte.
```

5.2 Cascade de Fournisseurs LLM

Afin de garantir la disponibilité du service, le système implémente une cascade de trois fournisseurs :

Priorité	Fournisseur	Modèle	Rôle
1	NVIDIA	llama-3.3-nemotron-super-49b	Principal
2	Groq	llama-3.3-70b-versatile	Fallback
3	DeepSeek	deepseek-chat	Fallback ultime

TABLE 4 – Cascade de fournisseurs LLM.

Paramètre de génération. — `temperature` = 0, garantissant le déterminisme des réponses (même question → même réponse).

5.3 Mémoire Conversationnelle

Le système utilise `ConversationBufferMemory` de LangChain pour conserver l'historique des échanges. Cette mémoire permet :

- Les questions de suivi implicites (*“Et dans quel fichier ?”* après *“Où est l'authentification ?”*) ;
- La cohérence du dialogue sur plusieurs tours ;
- L'adaptation du contexte en fonction de l'évolution de la conversation.

5.4 Citation des Sources

Chaque réponse est accompagnée d'un panneau **Sources** dans l'interface Streamlit, listant :

- Le nom du fichier source ;
- Les numéros de lignes concernés ;
- Un extrait du contenu pertinent.

Cette transparence permet à l'utilisateur de vérifier l'origine de chaque information.

6 Limitation des Hallucinations

L'anti-hallucination constitue le verrou le plus critique du système. Quatre mécanismes complémentaires sont déployés, par ordre de fiabilité :

6.1 Filtrage Cosine Large (Premier Passage)

Avant toute génération, le `RelevanceRetriever` vérifie la distance cosinus de chaque chunk. Seuil à 0,80 — volontairement large pour ne pas écarter de bons candidats trop tôt. On remonte 50 chunks. Si après ce filtre il reste 0 chunks, le LLM reçoit un contexte vide.

6.2 Cross-Encoder (Deuxième Passage, le Plus Précis)

C'est la pièce maîtresse de l'anti-hallucination. Le modèle `cross-encoder/ms-marco-MiniLM-L-6-v2` compare chaque paire (question, chunk) et donne un score logit. Contrairement au calcul cosinus qui compare des vecteurs, le cross-encoder lit réellement le texte et juge si le chunk répond à la question. Les chunks avec un score négatif sont rejetés.

6.3 Contrainte Comportementale par Prompt

Les instructions du prompt système (“réponds *UNIQUEMENT* sur le contexte”, “ne fais *AUCUNE* supposition”) orientent le comportement du LLM. C’est le dernier rempart — une instruction textuelle que le modèle peut théoriquement outrepasser.

6.4 Message de Fallback

En l’absence de contexte pertinent, la réponse par défaut est :

“Je ne dispose pas d’informations suffisantes dans le repository pour répondre.”

6.5 Limites Assumées

- **Pas de vérification post-hoc** : le système ne vérifie pas a posteriori si la réponse générée correspond au contexte fourni — ce serait la prochaine étape ;
- Le cross-encoder ajoute $\sim 1\text{--}2$ secondes de latence par requête, mais la qualité du filtrage compense largement ;
- La fonction `check_relevance()` définie dans le code est devenue obsolète depuis l’intégration du cross-encoder dans `RelevanceRetriever`.

Constat. — L’anti-hallucination repose sur 2 filtres (cosine + cross-encoder) avant même d’atteindre le LLM, puis le prompt en dernier recours. Les tests montrent que l’assistant refuse de répondre aux questions hors-sujet (exemple : “*Quel temps fait-il ?*”).

7 Identification des Zones Critiques du Projet

Le module `critical_analysis.py` effectue trois analyses indépendantes.

7.1 Fichiers Sensibles

Le système scanne chaque fichier indexé à la recherche de :

- Mots-clés : `password`, `secret`, `token`, `api_key`, `credentials` ;
- Patterns de tokens : chaînes de 32 caractères ou plus ;
- URLs : présence de `http://` ou `https://`.

Résultat sur Rafiki. — 281 occurrences potentielles identifiées, principalement des URLs et des mots-clés dans la documentation.

7.2 Complexité Cyclomatique

La librairie `radon` calcule la complexité de chaque fonction Python. Les fonctions dont la complexité dépasse 10 chemins d’exécution sont marquées comme complexes.

Résultat sur Rafiki. — 9 fonctions dépassent le seuil.

7.3 Modules Non Testés

Le système compare les modules présents dans `src/` avec les fichiers de test correspondants dans `tests/`. Les modules sans test sont listés.

Résultat sur Rafiki. — 27 modules sans test, ce qui est cohérent avec la nature documentaire et démonstrative du projet.

7.4 Accès Utilisateur

Les résultats des trois analyses sont indexés dans ChromaDB avec le tag `type='critical_analysis'`. L'utilisateur peut interroger l'assistant sur :

- “Quels sont les fichiers sensibles ?”
- “Montre-moi les zones complexes”
- “Quels modules n'ont pas de tests ?”

8 Évaluation de la Qualité de l'Assistant

8.1 Tests de Validation par Phase

Chaque phase de construction dispose de son script de validation :

Phase	Script	Objet
1 (Parsing)	<code>test_ingestion.py</code>	Vérifie la génération des chunks (812 unités)
2 (Indexation)	<code>test_embeddings.py</code>	Vérifie la vectorisation et le stockage ChromaDB
3 (RAG)	<code>test_rag.py</code>	Teste la chaîne complète avec questions pertinentes et pièges
4 (Critique)	<code>test_critical.py</code>	Vérifie la détection des zones sensibles, complexes, non testées

TABLE 5 – Scripts de validation par phase.

Un script `diagnostic_retrieval.py` permet en outre d'analyser le comportement du retriever à 3 niveaux (ChromaDB direct, wrapper LangChain, chaîne complète).

8.2 Évaluation Structurée

Un jeu de **20 questions** a été créé dans `evaluation/questions.json`, réparties en 5 catégories :

Catégorie	Nb	Objectif
Architecture	5	Structure globale, modules, frameworks
Fonctionnalités	5	Localisation de code, endpoints
Documentation	5	Description du projet, installation
Zones critiques	3	Fichiers sensibles, complexité, tests
Pièges hallucination	2	Questions sans réponse dans le repo

TABLE 6 – Jeu de questions d’évaluation structuré.

Le script `evaluate.py` pose chaque question automatiquement, enregistre la réponse, et calcule :

- **Taux de succès** : la réponse a-t-elle des sources et un contenu pertinent ?
- **Latence moyenne** : temps question → réponse ;
- **Taux d’anti-hallucination** : les questions pièges sont-elles bien refusées ?

8.3 Résultats des Tests

- Les questions retournent des sources **différentes** selon le sujet — plus de répétition systématique ;
- Plus d’erreur 413 depuis la réduction de la taille du contexte ;
- L’anti-hallucination (cosine + cross-encoder) bloque les questions hors-sujet ;
- Le cross-encoder ajoute $\sim 1\text{--}2\text{s}$ de latence mais améliore significativement la pertinence.

8.4 Axes d’Amélioration

- **Affinage des métriques** : le script `evaluate.py` existe mais les métriques devraient être plus fines (précision retrieval, taux d’hallucination formel) ;
- **Test de charge** : pas de validation avec 10 requêtes simultanées ;
- **Vérification humaine** : certaines réponses mériteraient une validation manuelle pour confirmer la justesse technique.

9 Usage de l’IA

L’utilisation de l’IA dans ce prototype se divise en deux parties principales.

9.1 Contributions de l’IA

9.1.1 Brainstorming et Conception

- Aide au débogage des erreurs et à la stabilisation de l’environnement de développement ;

- Structuration du prompt système anti-hallucination ;
- Rédaction du plan de développement et des checklists de suivi ;
- Exécution du prototype.

9.1.2 Usage du LLM dans le Prototype

- Vérification des chunks avec le cross-encoder avant l'envoi du prompt (deuxième niveau de filtrage) ;
- Envoi des chunks avec score supérieur au LLM pour identifier la meilleure réponse à la question posée.

9.2 Contributions Personnelles

9.2.1 Décisions Techniques

- Choix du chunking sémantique (par fonction ou classe) plutôt qu'un chunking fixe par nombre de lignes — des unités logiques, pas des blocs arbitraires ;
- Définition empirique des seuils : d'abord 0,65, puis passage à 0,80 après intégration du cross-encoder (le filtrage fin est délégué au cross-encoder) ;
- Choix du modèle multilingue après avoir constaté l'échec du modèle anglais sur les questions en français.

9.2.2 Implémentation

- Écriture complète du module `config.py` et de la logique de découverte et de filtrage ;
- Configuration de l'environnement (dépendances, `.env`, `.gitignore`) ;
- Tests manuels de retrieval et corrections itératives.

9.2.3 Analyse Critique

- Identification des lacunes en cours de route : `check_relevance()` rendue obsolète par le cross-encoder, seuil cosine à ajuster après ajout du cross-encoder, qualité du retrieval insuffisante avec le modèle anglais ;
- Décision consciente de prioriser un prototype fonctionnel ;
- Évaluation transparente des limites et des imperfections.

9.3 Limites de l'Aide IA

- **Domaine métier** : l'IA ne connaît pas le contexte pédagogique du projet Rafiki (accompagnement scolaire) et propose parfois des solutions génériques inadaptées ;
- **Bugs subtils** : le code généré peut contenir des erreurs (exemple : `check_relevance()` définie mais jamais appelée — rendue obsolète par le cross-encoder, mais découverte seulement en relecture critique) ;
- **Dépendances obsolètes** : les suggestions de librairies nécessitent une vérification (corrections apportées pour LangChain 0.3+) ;
- **Validation humaine** : l'IA ne peut pas évaluer la pertinence réelle des réponses — cette étape reste un travail humain indispensable.

10 Conclusion

10.1 Forces du Prototype

- Architecture modulaire et évolutive ;
- Anti-hallucination à 2 niveaux (filtrage cosine + cross-encoder) + prompt comportemental ;
- Transparence totale sur les sources (fichier + lignes) ;
- Cascade de fournisseurs garantissant la disponibilité ;
- Honnêteté sur les lacunes et les limites.

10.2 Axes d'Amélioration

- Vérification post-hoc des réponses générées ;
- Test de charge (10+ requêtes simultanées) ;
- Affinage des métriques d'évaluation (précision retrieval, taux d'hallucination formel) ;
- Génération de rapports `.md` pour l'onboarding des nouveaux développeurs.

Document rédigé et développé par ELOUAKATE Saad.

Juin 2026